SchoolConnect – Full-Stack School Social & Academic Platform

Google AI Studio Master System Prompt & Architectural Specification

1. Executive Overview

SchoolConnect is a production-grade, full-stack educational community platform designed specifically for school environments. It unifies modern, safe student social interaction with a rigorous **Academic Suite** (untimed quiz competitions, school-wide leaderboards, homework assignment management, and a collaborative study notes repository).

The platform bridges the gap between social engagement and academic excellence in a closed, moderated, school-first environment featuring fine-grained Role-Based Access Control (RBAC) across **Students**, **Teachers/Faculty**, and **Super Administrators**.

2. Core Functional Modules

A. Authentication & Account Security

- **Email Registration & Login**: Registration with school email, full name, role selection, class, and section.
- **JWT Bearer Token Authentication**: Secure token issuance (`7d` validity), authorization headers (`Bearer <token>`), and persistent login state.
- **Password Security**: Bcrypt hashing (salt rounds: 12).
- **Password Reset Flow**: Cryptographically secure token generation (`crypto.randomBytes`), expiring in 1 hour, delivered via email/Nodemailer with verification endpoints.
- **Email Verification**: Account activation token workflow ensuring genuine student identity.
- **Account Moderation & Locking**: Real-time `isBlocked` flag check on every authenticated request.

B. User Profiles & Digital Identity

- Profile avatar upload and custom cover banner.
- Academic metadata: Class/Grade (e.g., *Grade 9*), Section (e.g., *Section A*).
- Personal Bio, Interests & Hobbies tags, contact details.
- User activity overview: Posts published, clubs joined, competitions taken, assignments turned in, and helpful notes shared.

C. Safe Social Feed & Student Networking

- **Post Creation**: Rich text posts with optional image attachments and hashtag parsing (`#science` , `#sports`).
- **Engagements**: Instant likes with counter, nested comments, and author badge display.
- **Club-Linked Posts**: Posts can be scoped to specific school clubs or broadcast school-wide.
- **Content Moderation**: Flagging system (`Report` table) enabling students to report inappropriate content directly to teachers and administrators.

D. School Clubs & Communities

- Pre-seeded and custom extracurricular clubs:
- 🔬 **Science Club** (Research, experiments, science fairs)
- 🌿 **Eco Club** (Sustainability, green campus initiatives)
- 💻 **Coding & Robotics Club** (Hackathons, programming projects)
- ⚽ **Sports Club** (Athletics, intramural tournaments)
- 🎨 **Arts & Culture Club** (Creative arts, exhibitions, theatre)
- Dedicated club pages with member rosters, discussion boards, and scheduled calendar events with dates and campus venues.
- Club leadership hierarchy: `MEMBER` , `MODERATOR` , and `LEADER` .

E. Real-Time One-to-One Messaging

- Low-latency chat powered by **Socket.IO** WebSockets.
- Real-time online/offline presence indicators.
- Instant delivery of private messages between mutual connections.
- Persistent conversation history stored in PostgreSQL with read receipts (`isRead`).

F. Interactive Polls & School Voting

- Single-choice polls created by students, teachers, or administrators.
- Strict one-vote-per-student database constraint (`@@unique([pollId, userId])`).
- Real-time visual percentage progress bars and total vote counts.
- Optional expiration timestamps and auto-closure.

G. Official School Notice Board

- Authoritative announcements published exclusively by **Teachers** and **Administrators**.
- Categorized circulars:
- 📋 `CIRCULAR` (Official school circulars and administrative letters)
- 📅 `EXAM_SCHEDULE` (Mid-term, terminal, and Olympiad timetables)
- 🎪 `EVENT` (Annual day, sports day, parent-teacher meetings)
- 🏖️ `HOLIDAY_NOTICE` (Government and school holiday declarations)
- Urgency badge priority: `NORMAL` , `IMPORTANT` , and `URGENT` .

- Grade-level targeting (e.g., broadcast to `ALL` or specific grades).
- Downloadable document attachments.

---

3. Academic Suite (New Core Architecture)

1. Quizzes & Olympiad Competitions (`/quizzes`)

- **Untimed Competition Philosophy**: No stress clocks or countdown timers. Students complete questions thoughtfully, review answers, and submit when confident.
- **Scheduled Competition Lifecycle**:
- `startDate` : Timestamp when the quiz officially opens for participation.
- `endDate` : Timestamp when the quiz closes and official results are revealed.
- Real-time status tags: 🟢 **Live Contest**, ⏳ **Upcoming**, 🏆 **Results Out**.
- **Automated Grading & Integrity**:
- Objective multiple-choice question format (4 options with single correct answer).
- One submission per student constraint (`@@unique([quizId, studentId])`).
- Score and correct-answer lockdown: Correct answers remain hidden until after `endDate` to preserve competition integrity.
- **Role Permission**: Only **Teachers** and **Administrators** can conduct new quiz competitions (`POST /api/v1/quizzes`) and remove competitions (`DELETE /api/v1/quizzes/:id`).

2. Full Participant Leaderboard & Winners Podium (`/leaderboard`)

- **Global & Competition-Specific Rankings**: Toggle between individual competition standings and the overall School Hall of Fame.
- **Top 3 Winners Podium**: Dedicated visual showcase celebrating Gold 🥇 (1st Place), Silver 🥈 (2nd Place), and Bronze 🥉 (3rd Place).
- **Exhaustive Participant Standings**: Displays **all participants** (Rank 1 to N), including student name, class/grade, points earned, score percentage, and completion timestamp.
- Search and grade-filter capabilities to easily look up classmates and rankings.

3. Assignments & Homework Portal (`/assignments`)

- **Teacher Assignment Creation**: Set title, description, subject, target grade, max points, reference attachment, and due date.
- **Student Online Submissions**: Students turn in their work before deadlines with written responses and cloud-hosted attachments (Google Drive, PDFs, image proofs).
- **Teacher Grading & Feedback Drawer**:
- Review all submitted student works in real-time.
- Assign numeric points (e.g., 95/100) and automatic letter grades (A+, A, B, C).
- Provide personalized, constructive teacher feedback.

- Status filters: `All`, `Pending Submission`, `Submitted`, and `Graded`.

4. Study Notes & Revision Hub (`/notes`)

- Centralized digital library for revision notes, formula sheets, chapter summaries, and past papers.
- Tagged by Subject (*Mathematics, Science, Coding, English, Social Studies*) and Class/Grade (*Grade 6 to 12*).
- Direct document preview and external link access (Google Drive, PDFs).
- **Helpful Vote System (👍)**: Students and teachers upvote useful revision notes, dynamically ranking the most helpful resources at the top.

---

4. User Roles & Permission Matrix (RBAC)

Capability / Action	Student	Teacher / Faculty	Administrator
Register & Manage Personal Profile	✅	✅	✅
Create Posts, Comments & Likes	✅	✅	✅
Join Clubs & Participate in Events	✅	✅	✅
Chat with Friends (1-to-1)	✅	✅	✅
Vote in School Polls	✅	✅	✅
Take Untimed Quizzes & Competitions	✅	✅	✅
View Full Leaderboards & Podiums	✅	✅	✅
Submit Homework Assignments	✅	❌	❌
Upvote Helpful Study Notes	✅	✅	✅
Conduct / Publish New Quizzes	❌	✅	✅
Delete Quiz Competitions	❌	✅	✅
Create Homework Assignments	❌	✅	✅
Grade Submissions & Give Feedback	❌	✅	✅
Upload Study Notes to Hub	✅	✅	✅
Publish Official School Notices	❌	✅	✅
Block / Unblock Users	❌	❌	✅
Moderate / Delete Any Post	❌	✅	✅
Delete Clubs & Manage System Metrics	❌	❌	✅

Configured Super Administrator: `nkms143@gmail.com` (`E.Amarnath`)

---

5. Technology Stack & Architectural Patterns

Frontend Architecture

- **Framework**: React 18 with TypeScript 5
- **Tooling & Bundler**: Vite 5 (Fast HMR, optimized chunking, ~20s production build)
- **Styling**: Tailwind CSS v3/4 with dark mode support and custom color tokens
- **Routing**: React Router DOM v6 (Protected routes, role gates, SPA rewrites)
- **Icons**: Lucide React
- **HTTP Client**: Axios with global interceptors for JWT token injection and 401 handling
- **Real-Time Client**: Socket.IO Client for bidirectional event listeners

Backend Architecture

- **Runtime**: Node.js 20+ (ES Modules)
- **Framework**: Express.js 5 with TypeScript
- **Real-Time Engine**: Socket.IO server bound to HTTP server
- **Database ORM**: Prisma ORM 5 with dual client generators (root & server client)
- **Security & Utilities**:
- `bcryptjs` : Password hashing
- `jsonwebtoken` : Stateless access token authentication
- `cors` : Allowed origins with dynamic regex supporting Render deployments
- `helmet` : HTTP security headers
- `express-rate-limit` : Brute-force and DDoS mitigation
- `nodemailer` : SMTP email transport for account verification and password recovery

Database Architecture

- **Database**: PostgreSQL 15 hosted on **Supabase** (AWS Mumbai `ap-south-1` region)
- **Connection Strategy**: Supabase Session/Transaction connection pooling via port `6543` / `5432` with SSL mode `require`
- **Schema Management**: Prisma Schema (`prisma/schema.prisma`) with declarative push (`npx prisma db push`)

---

6. PostgreSQL Database Schema (Prisma Models)

```
datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

generator serverClient {
  provider = "prisma-client-js"
  output   = "../server/node_modules/.prisma/client"
}

enum Role {
  STUDENT
  TEACHER
  ADMIN
}

enum NoticeCategory {
  CIRCULAR
  EXAM_SCHEDULE
  EVENT
  HOLIDAY_NOTICE
}

enum NoticePriority {
  NORMAL
  IMPORTANT
  URGENT
}

enum FriendRequestStatus {
  PENDING
  ACCEPTED
  REJECTED
}

enum ClubRole {
  MEMBER
  MODERATOR
  LEADER
}

enum ReportStatus {
  PENDING
  RESOLVED
  DISMISSED
}

enum QuizStatus {
  DRAFT
  PUBLISHED
  CLOSED
```

```
}

enum SubmissionStatus {
  PENDING
  GRADED
  LATE
}

// ------------------------------------------------------------
// CORE USERS & SOCIAL MODELS
// ------------------------------------------------------------
model User {
  id                    String              @id @default(cuid())
  email                 String              @unique
  password              String
  role                  Role                @default(STUDENT)
  isVerified            Boolean             @default(false)
  verificationToken     String?
  resetPasswordToken    String?
  resetPasswordExpires  DateTime?
  isBlocked             Boolean             @default(false)
  createdAt             DateTime            @default(now())
  updatedAt             DateTime            @updatedAt

  profile               Profile?
  posts                 Post[]              @relation("UserPosts")
  comments              Comment[]
  likes                 Like[]
  sentFriendRequests    FriendRequest[]     @relation("SentRequests")
  receivedFriendRequests FriendRequest[]    @relation("ReceivedRequests")
  friendshipsInitiated  Friendship[]        @relation("UserAFriends")
  friendshipsReceived   Friendship[]        @relation("UserBFriends")
  clubMemberships       ClubMember[]
  ledClubs              Club[]              @relation("ClubLeader")
  clubDiscussions       ClubDiscussion[]
  createdPolls          Poll[]
  votes                 Vote[]
  sentMessages          Message[]           @relation("SentMessages")
  receivedMessages      Message[]           @relation("ReceivedMessages")
  announcements         Announcement[]
  reportsSubmitted      Report[]            @relation("Reporter")

  // Academic Suite
  quizzesCreated        Quiz[]              @relation("QuizCreator")
  quizSubmissions       QuizSubmission[]
  assignmentsCreated    Assignment[]        @relation("AssignmentCreator")
  assignmentSubmissions AssignmentSubmission[]
  notesAuthored         StudyNote[]         @relation("NoteAuthor")
  noteHelpfulVotes      NoteHelpfulVote[]

  @@index([email])
  @@index([role])
}

model Profile {
  id          String   @id @default(cuid())
  userId      String   @unique
  user        User     @relation(fields: [userId], references: [id], onDelete: Cascade)
```

```
  fullName    String
  avatarUrl   String?
  grade       String?
  section     String?
  bio         String?
  interests   String?
  phone       String?
  coverImage  String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model Post {
  id          String    @id @default(cuid())
  authorId    String
  author      User      @relation("UserPosts", fields: [authorId], references: [id], onDelete:
Cascade)
  content     String
  imageUrl    String?
  clubId      String?
  club        Club?     @relation(fields: [clubId], references: [id], onDelete: SetNull)
  hashtags    String?
  isModerated Boolean   @default(false)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  comments    Comment[]
  likes       Like[]

  @@index([authorId])
  @@index([createdAt])
  @@index([clubId])
}

model Comment {
  id        String   @id @default(cuid())
  postId    String
  post      Post     @relation(fields: [postId], references: [id], onDelete: Cascade)
  authorId  String
  author    User     @relation(fields: [authorId], references: [id], onDelete: Cascade)
  content   String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([postId])
  @@index([authorId])
}

model Like {
  id        String   @id @default(cuid())
  postId    String
  post      Post     @relation(fields: [postId], references: [id], onDelete: Cascade)
  userId    String
  user      User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  createdAt DateTime @default(now())

  @@unique([postId, userId])
  @@index([postId])
```

```
  @@index([userId])
}

model FriendRequest {
  id          String              @id @default(cuid())
  senderId    String
  sender      User                @relation("SentRequests", fields: [senderId], references:
[id], onDelete: Cascade)
  receiverId  String
  receiver    User                @relation("ReceivedRequests", fields: [receiverId],
references: [id], onDelete: Cascade)
  status      FriendRequestStatus @default(PENDING)
  createdAt   DateTime            @default(now())
  updatedAt   DateTime            @updatedAt

  @@unique([senderId, receiverId])
  @@index([senderId])
  @@index([receiverId])
}

model Friendship {
  id        String   @id @default(cuid())
  userAId   String
  userA     User     @relation("UserAFriends", fields: [userAId], references: [id], onDelete:
Cascade)
  userBId   String
  userB     User     @relation("UserBFriends", fields: [userBId], references: [id], onDelete:
Cascade)
  createdAt DateTime @default(now())

  @@unique([userAId, userBId])
  @@index([userAId])
  @@index([userBId])
}

model Club {
  id          String           @id @default(cuid())
  name        String           @unique
  slug        String           @unique
  description String
  category    String
  bannerUrl   String?
  iconUrl     String?
  leaderId    String?
  leader      User?            @relation("ClubLeader", fields: [leaderId], references: [id],
onDelete: SetNull)
  createdAt   DateTime         @default(now())
  updatedAt   DateTime         @updatedAt

  members     ClubMember[]
  posts       Post[]
  events      ClubEvent[]
  discussions ClubDiscussion[]
}

model ClubMember {
  id       String   @id @default(cuid())
  clubId   String
```

```
  club     Club     @relation(fields: [clubId], references: [id], onDelete: Cascade)
  userId   String
  user     User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  role     ClubRole @default(MEMBER)
  joinedAt DateTime @default(now())

  @@unique([clubId, userId])
  @@index([clubId])
  @@index([userId])
}

model ClubEvent {
  id          String   @id @default(cuid())
  clubId      String
  club        Club     @relation(fields: [clubId], references: [id], onDelete: Cascade)
  title       String
  description String
  eventDate   DateTime
  location    String
  createdAt   DateTime @default(now())

  @@index([clubId])
}

model ClubDiscussion {
  id        String   @id @default(cuid())
  clubId    String
  club      Club     @relation(fields: [clubId], references: [id], onDelete: Cascade)
  authorId  String
  author    User     @relation(fields: [authorId], references: [id], onDelete: Cascade)
  title     String
  content   String
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  @@index([clubId])
  @@index([authorId])
}

model Poll {
  id        String       @id @default(cuid())
  creatorId String
  creator   User         @relation(fields: [creatorId], references: [id], onDelete: Cascade)
  question  String
  expiresAt DateTime?
  isClosed  Boolean      @default(false)
  createdAt DateTime     @default(now())
  updatedAt DateTime     @updatedAt

  options   PollOption[]
  votes     Vote[]

  @@index([creatorId])
}

model PollOption {
  id        String   @id @default(cuid())
  pollId    String
```

```
  poll      Poll     @relation(fields: [pollId], references: [id], onDelete: Cascade)
  text      String
  votes     Vote[]

  @@index([pollId])
}

model Vote {
  id        String     @id @default(cuid())
  pollId    String
  poll      Poll       @relation(fields: [pollId], references: [id], onDelete: Cascade)
  optionId  String
  option    PollOption @relation(fields: [optionId], references: [id], onDelete: Cascade)
  userId    String
  user      User       @relation(fields: [userId], references: [id], onDelete: Cascade)
  createdAt DateTime   @default(now())

  @@unique([pollId, userId])
  @@index([pollId])
  @@index([userId])
}

model Message {
  id         String   @id @default(cuid())
  senderId   String
  sender     User     @relation("SentMessages", fields: [senderId], references: [id], onDelete:
Cascade)
  receiverId String
  receiver   User     @relation("ReceivedMessages", fields: [receiverId], references: [id],
onDelete: Cascade)
  content    String
  isRead     Boolean  @default(false)
  createdAt  DateTime @default(now())

  @@index([senderId, receiverId])
  @@index([receiverId])
}

model Announcement {
  id            String         @id @default(cuid())
  authorId      String
  author        User           @relation(fields: [authorId], references: [id], onDelete:
Cascade)
  title         String
  content       String
  category      NoticeCategory @default(CIRCULAR)
  priority      NoticePriority @default(NORMAL)
  attachmentUrl String?
  targetGrade   String?        @default("ALL")
  expiresAt     DateTime?
  createdAt     DateTime       @default(now())
  updatedAt     DateTime       @updatedAt

  @@index([category])
  @@index([priority])
  @@index([createdAt])
}
```

```
model Report {
  id          String       @id @default(cuid())
  reporterId  String
  reporter    User         @relation("Reporter", fields: [reporterId], references: [id],
onDelete: Cascade)
  targetType  String
  targetId    String
  reason      String
  status      ReportStatus @default(PENDING)
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt

  @@index([status])
  @@index([reporterId])
}

// ------------------------------------------------------------
// ACADEMIC SUITE MODELS
// ------------------------------------------------------------
model Quiz {
  id                String           @id @default(cuid())
  creatorId         String
  creator           User             @relation("QuizCreator", fields: [creatorId], references:
[id], onDelete: Cascade)
  title             String
  description       String
  subject           String
  grade             String
  totalPoints       Int              @default(100)
  status            QuizStatus       @default(PUBLISHED)
  startDate         DateTime
  endDate           DateTime
  isResultsReleased Boolean          @default(false)
  createdAt         DateTime         @default(now())
  updatedAt         DateTime         @updatedAt

  questions         QuizQuestion[]
  submissions       QuizSubmission[]

  @@index([creatorId])
  @@index([subject])
  @@index([grade])
  @@index([startDate, endDate])
}

model QuizQuestion {
  id           String       @id @default(cuid())
  quizId       String
  quiz         Quiz         @relation(fields: [quizId], references: [id], onDelete: Cascade)
  questionText String
  options      String[]
  correctIndex Int
  points       Int          @default(10)
  order        Int          @default(0)

  answers      QuizAnswer[]

  @@index([quizId])
```

```
}

model QuizSubmission {
  id          String       @id @default(cuid())
  quizId      String
  quiz        Quiz         @relation(fields: [quizId], references: [id], onDelete: Cascade)
  studentId   String
  student     User         @relation(fields: [studentId], references: [id], onDelete: Cascade)
  score       Int          @default(0)
  maxScore    Int          @default(0)
  completedAt DateTime     @default(now())

  answers     QuizAnswer[]

  @@unique([quizId, studentId])
  @@index([quizId, score, completedAt])
  @@index([studentId])
}

model QuizAnswer {
  id            String         @id @default(cuid())
  submissionId  String
  submission    QuizSubmission @relation(fields: [submissionId], references: [id], onDelete:
Cascade)
  questionId    String
  question      QuizQuestion   @relation(fields: [questionId], references: [id], onDelete:
Cascade)
  selectedIndex Int
  isCorrect     Boolean
  pointsAwarded Int            @default(0)

  @@index([submissionId])
}

model Assignment {
  id            String                @id @default(cuid())
  creatorId     String
  creator       User                  @relation("AssignmentCreator", fields: [creatorId],
references: [id], onDelete: Cascade)
  title         String
  description   String
  subject       String
  grade         String
  dueDate       DateTime
  maxPoints     Int                   @default(100)
  attachmentUrl String?
  createdAt     DateTime              @default(now())
  updatedAt     DateTime              @updatedAt

  submissions   AssignmentSubmission[]

  @@index([creatorId])
  @@index([subject])
  @@index([grade])
  @@index([dueDate])
}

model AssignmentSubmission {
```

```
  id            String           @id @default(cuid())
  assignmentId  String
  assignment    Assignment       @relation(fields: [assignmentId], references: [id], onDelete:
Cascade)
  studentId     String
  student       User             @relation(fields: [studentId], references: [id], onDelete:
Cascade)
  content       String
  attachmentUrl String?
  status        SubmissionStatus @default(PENDING)
  grade         Int?
  feedback      String?
  submittedAt   DateTime         @default(now())
  gradedAt      DateTime?

  @@unique([assignmentId, studentId])
  @@index([assignmentId])
  @@index([studentId])
}

model StudyNote {
  id           String           @id @default(cuid())
  authorId     String
  author       User             @relation("NoteAuthor", fields: [authorId], references: [id],
onDelete: Cascade)
  title        String
  description  String
  subject      String
  grade        String
  chapter      String?
  fileUrl      String
  fileType     String
  helpfulCount Int              @default(0)
  createdAt    DateTime         @default(now())
  updatedAt    DateTime         @updatedAt

  helpfulVotes NoteHelpfulVote[]

  @@index([subject])
  @@index([grade])
  @@index([authorId])
}

model NoteHelpfulVote {
  id        String    @id @default(cuid())
  noteId    String
  note      StudyNote @relation(fields: [noteId], references: [id], onDelete: Cascade)
  userId    String
  user      User      @relation(fields: [userId], references: [id], onDelete: Cascade)
  createdAt DateTime  @default(now())

  @@unique([noteId, userId])
}
```

7. Complete REST API Specification

Authentication (`/api/v1/auth`)

- `POST /register` : Create student/teacher account with hashed password.
- `POST /login` : Authenticate credentials, verify status, return JWT and user profile.
- `GET /me` : Return current authenticated session profile.
- `POST /forgot-password` : Send 1-hour secure password reset token.
- `POST /reset-password` : Set new password with cryptographic token.
- `GET /verify-email` : Verify student email address.

Quizzes & Competitions (`/api/v1/quizzes`)

- `GET /` : Fetch all competitions with user-submission status and dynamic state (`ACTIVE` , `UPCOMING` , `ENDED`).
- `GET /:id` : Fetch competition questions (omits `correctIndex` if user is taking the quiz or contest is ongoing).
- `POST /` : *(Teachers & Admins)* Create new competition with untimed question pool and schedule.
- `POST /:id/submit` : Grade student answers, calculate total score, record submission.
- `GET /:id/leaderboard` : Fetch ranking of all participants for a specific competition.
- `GET /leaderboard/global` : Fetch school-wide Hall of Fame overall student leaderboard.
- `DELETE /:id` : *(Teachers & Admins)* Delete quiz and related submissions.

Assignments & Homework (`/api/v1/assignments`)

- `GET /` : Fetch assignments filtered by subject, grade, and submission status.
- `GET /:id` : Fetch assignment details with student submission state.
- `POST /` : *(Teachers & Admins)* Post new homework assignment with due date and attachments.
- `POST /:id/submit` : Submit student work (text summary + cloud drive link).
- `GET /:id/submissions` : *(Teachers & Admins)* Retrieve all student turned-in assignments.
- `POST /submissions/:submissionId/grade` : *(Teachers & Admins)* Grade student work with points (e.g. 95/100) and feedback.

Study Notes (`/api/v1/notes`)

- `GET /` : Retrieve shared study notes with subject/grade filters and search.
- `POST /` : Upload new revision guide, chapter summary, or formula sheet.
- `POST /:id/helpful` : Toggle helpful upvote (👍) on a study guide.
- `DELETE /:id` : Remove note (author or admin only).

Posts & Feeds (`/api/v1/posts`)

- `GET /` : Feed of recent school posts with comment/like counts.

- `POST /` : Create new post (optional image and hashtags).
- `POST /:id/like` : Toggle like on post.
- `POST /:id/comments` : Add comment to post.
- `DELETE /:id` : Remove post (author, teacher, or admin).

Clubs (`/api/v1/clubs`)

- `GET /` : List all school clubs with category and membership counts.
- `GET /:slug` : Retrieve club profile, discussions, events, and member list.
- `POST /:slug/join` : Join club.
- `POST /:slug/leave` : Leave club.
- `POST /:slug/events` : *(Club Leader / Teacher)* Create club event.
- `POST /:slug/discussions` : Start new discussion thread in club.

Polls (`/api/v1/polls`)

- `GET /` : List all active and past polls with user vote indicators.
- `POST /` : Create new school poll.
- `POST /:id/vote` : Submit single choice vote on poll option.

Notice Board (`/api/v1/announcements`)

- `GET /` : Fetch official school notices filtered by category and priority.
- `POST /` : *(Teachers & Admins)* Publish official circular or exam schedule.
- `DELETE /:id` : *(Admins & Authors)* Delete notice.

Direct Messaging (`/api/v1/messages`)

- `GET /conversations` : List active chats with latest message and unread count.
- `GET /:userId` : Retrieve message history between two users.
- `POST /:userId` : Send direct message (also emits real-time WebSocket event).

---

8. Deployment Architecture

High-Level Topology

```
flowchart TD
    User["👨‍🎓 Students & 👩‍🏫 Teachers (Web Browsers)"]

    subgraph Render["Render Cloud Platform"]
        Frontend["🌐 Static Site: schoolconnect (React / Vite SPA)\nhttps://schoolconnect-3gwn.onrender.com\nRewrite: /* → /index.html"]
        Backend["⚙️ Web Service: schoolconnect-backend (Node.js / Express)\nhttps://schoolconnect-backend-1bs9.onrender.com\nPort 10000 | Health: /api/health"]
    end

    subgraph CloudDB["Supabase Managed Cloud"]
        Postgres["🐘 PostgreSQL 15 Database\naws-0-ap-south-1.pooler.supabase.com\nSession Pooler (Port 5432/6543)"]
    end

    User ⟶|HTTPS Traffic| Frontend
    Frontend ⟶|REST API Requests| Backend
    Frontend ⟷|WebSockets (Socket.IO)| Backend
    Backend ⟶|Prisma ORM over TLS| Postgres
```

1. Live Production Endpoints

- **Frontend URL**: `https://schoolconnect-3gwn.onrender.com`
- **Backend API URL**: `https://schoolconnect-backend-1bs9.onrender.com`
- **Health Check**: `https://schoolconnect-backend-1bs9.onrender.com/api/health`
- **Git Repository**: `https://github.com/nkms143/schoolconnect.git` (`main` branch)

2. Render Blueprint Specification (`render.yaml`)

```yaml
version: "1"

services:
  # Backend Node.js / Express API Service
  - type: web
    name: schoolconnect-backend
    runtime: node
    plan: free
    region: oregon
    rootDir: server
    buildCommand: npm install --include=dev && npm run db:generate && npm run build
    startCommand: npm run db:push && npm run db:seed && npm start
    healthCheckPath: /api/health
    envVars:
      - key: NODE_ENV
        value: production
      - key: PORT
        value: 10000
      - key: DATABASE_URL
        sync: false
      - key: JWT_SECRET
        generateValue: true
      - key: JWT_EXPIRES_IN
        value: 7d
      - key: CLIENT_URL
        value: "*"

  # Frontend React Static Site
  - type: web
    name: schoolconnect
    runtime: static
    rootDir: client
    buildCommand: npm install && npm run build
    staticPublishPath: dist
    routes:
      - type: rewrite
        source: /*
        destination: /index.html
    envVars:
      - key: VITE_API_URL
        fromService:
          type: web
          name: schoolconnect-backend
          property: host
      - key: VITE_SOCKET_URL
        fromService:
          type: web
          name: schoolconnect-backend
          property: host
```

3. Environment Variables Reference

Backend Service (`server/.env`)

Variable	Required	Description	Example / Value
`NODE_ENV`	Yes	Application environment	`production`
`PORT`	Yes	HTTP server binding port	`10000`
`DATABASE_URL`	Yes	PostgreSQL connection pooler URI	`postgresql://postgres.[ref]:[pwd]@aws-0-ap-south-1.pooler.supabase.com:6543/postgres?pgbouncer=true`
`JWT_SECRET`	Yes	Secret key for signing bearer tokens	64-char random hex string
`JWT_EXPIRES_IN`	Yes	Token expiry duration	`7d`
`CLIENT_URL`	Yes	Allowed CORS origin (or wildcard)	`https://schoolconnect-3gwn.onrender.com`
`SMTP_HOST`	Optional	SMTP host for password resets	`smtp.gmail.com`
`SMTP_PORT`	Optional	SMTP port	`587`
`SMTP_USER`	Optional	SMTP username	`schoolconnect.mailer@gmail.com`
`SMTP_PASS`	Optional	SMTP application password	`app_password_here`

Frontend Static Site (`client/.env`)

Variable	Required	Description	Example / Value
`VITE_API_URL`	Yes	Target backend REST API URL	`https://schoolconnect-backend-1bs9.onrender.com`
`VITE_SOCKET_URL`	Yes	Target backend Socket.IO URL	`https://schoolconnect-backend-1bs9.onrender.com`

4. Monorepo Directory Layout

```
schoolconnect/
├── client/                          # Frontend React SPA
│   ├── src/
│   │   ├── components/
│   │   │   ├── common/              # Navbar, Sidebar, MobileDrawer, Button, Modal
│   │   │   ├── quizzes/             # QuizTakeModal, QuizCreateModal
│   │   │   ├── assignments/         # AssignmentSubmitModal, CreateModal, GradeDrawer
│   │   │   ├── notes/               # NoteUploadModal
│   │   │   ├── clubs/               # ClubCard, EventCard, DiscussionModal
│   │   │   └── posts/               # PostCard, CreatePostCard, CommentSection
│   │   ├── context/                 # AuthContext, ThemeContext, SocketContext
│   │   ├── pages/                   # QuizzesPage, Leaderboard, Assignments, Notes, etc.
│   │   ├── services/                # Axios API service abstraction modules
│   │   ├── types/                   # TypeScript models and API payload contracts
│   │   ├── App.tsx                  # Client routes and protected route gates
│   │   └── main.tsx                 # React root mount
│   ├── index.html
│   ├── package.json
│   ├── tsconfig.json
│   └── vite.config.ts
├── server/                          # Backend Express REST & WebSocket API
│   ├── src/
│   │   ├── config/                  # Environment parser and Prisma client instances
│   │   ├── controllers/             # Controllers (Quiz, Assignment, Note, Auth, etc.)
│   │   ├── middleware/              # authMiddleware, requireRole, errorHandler
│   │   ├── routes/                  # Express router definitions mounted under /api/v1
│   │   ├── services/                # Socket.IO handlers, email transport
│   │   ├── app.ts                   # Express application factory and middleware pipeline
│   │   └── server.ts                # HTTP and WebSocket server initialization
│   ├── package.json
│   └── tsconfig.json
├── prisma/
│   ├── schema.prisma                # PostgreSQL schema definition with dual generators
│   └── seed.ts                      # Comprehensive database seeding script
├── scripts/
│   └── seed-academic-data.ts        # Academic Suite sample seed script
├── docs/
│   └── DEPLOYMENT_GUIDE.md          # Detailed step-by-step production deployment manual
├── render.yaml                      # Render infrastructure blueprint specification
├── SchoolConnect_AI_Studio_Prompt.md  # Master AI Studio System Prompt & Specification
└── README.md                        # High-level project documentation
```

9. Instructions for Google AI Studio

When prompted with this specification, Google AI Studio must: 1. **Adhere to the Separation of Concerns**: Maintain isolated, type-safe API controllers, unified Prisma schema definitions, and modular React components. 2. **Preserve Untimed Quiz Rules**: Never inject forced countdown timers or auto-submission clocks into competition models or components. Results and correct answer options must remain concealed until after `endDate`. 3. **Respect Role Privileges (RBAC)**: Ensure that destructive or administrative actions (`POST /quizzes` , `POST /assignments` , `POST /announcements` , grading workflows,

user blocking) are strictly protected by server-side role validation middleware. 4. **Maintain Production Styling & High Contrast**: Ensure all buttons and action cards render with accessible color contrast ratios across light and dark modes (e.g., preventing white text on white backgrounds). 5. **Ensure Render & Supabase Compatibility**: Generate dual Prisma client generators so that monorepo paths (`client/` and `server/`) resolve generated types without compilation errors.